

Reinforcement Learning Lab

Lesson 12: SAC

Alberto Castellini and Riccardo Fidanza

University of Verona

email: alberto.castellini@univr.it, riccardo.fidanza@studenti.univr.it

Academic Year 2025-26



UNIVERSITÀ
di VERONA
Dipartimento
di **INFORMATICA**

Environment Setup (CleanRL) - 1/3

We recommend using **Python 3.10** or **Python 3.11**. Create a virtual environment to isolate the project dependencies.

- **Conda (Recommended)**

```
conda create -n cleanrl python=3.11
conda activate cleanrl
```

- **venv (Unix / Linux / macOS)**

```
python3 -m venv .venv
source .venv/bin/activate
```

- **venv (Windows)**

```
python -m venv .venv
.\.venv\Scripts\activate
```

Note: If PowerShell blocks execution, run `Set-ExecutionPolicy RemoteSigned -Scope CurrentUser` first.

Environment Setup (CleanRL) - 2/3

Once the environment is active, install the necessary dependencies.

- **Method A: Using requirements.txt (file in folder tools)**

```
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

- **Method B: Manual Installation (Alternative)**

If *requirements.txt* causes conflicts, you can manually install the core packages needed for CleanRL:

```
python -m pip install --upgrade pip  
pip install torch numpy gymnasium matplotlib tensorboard setuptools
```

- **Method C: Manual Installation (conda)**

(Recommended if you created the environment with Conda)

```
conda install pytorch numpy gymnasium matplotlib tensorboard setuptools -c pytorch
```

Notice: Remember to run these commands *inside* the activated `cleanrl` or `venv` environment.

Environment Setup (CleanRL) - 3/3

Update the repository of the lab:

```
cd RL-Lab
git stash
git pull
git stash pop
```

Training the RL Agent

You can train the agents using the provided scripts. These will automatically save the models and learning curves in the `runs/` folder.

- **Windows:** `.\lesson12_cleanRL_sac_run_train.bat`
- **Unix / macOS:** `chmod +x lesson12_cleanRL_sac_run_train.sh` then `./lesson12_cleanRL_sac_run_train.sh`

Important Notes for the Lab:

- **Hyperparameters:** Scripts run with default values. You can override them manually from the terminal. Check the argument parsing section in the Python script to see all available options. Example:

```
python lesson12_cleanRL_sac_train_code.py --env-id Pendulum-v1 --total-timesteps 50
```

- **Batch Execution:** Inside the `.bat` or `.sh` files, you can modify the code to run a single environment or loop through a list of them.
- **Recommendation:** We strongly advise training **one environment at a time!**

CleanRL: Code Flow

CleanRL is known for its **single-file implementation** approach. The execution flow is structured as follows:

- **Execution Wrappers (.bat or .sh):** Scripts used to launch the training process, allowing you to easily run single or multiple environments in a row.
- **Training Script (lesson12_cleanRL_sac_train_code.py):** A fully self-contained file managing the entire RL pipeline:
 - ▶ *parse_args()*: Defines and parses all the hyperparameters.
 - ▶ *Agent Class*: Contains the neural network definitions (both Actor and Critic) and the action-sampling logic.
 - ▶ *Main Loop*: Sets up vectorized environments, collects rollouts, computes Advantages, and performs the SAC network updates (**this is where you will implement the missing equations!**).
- **Testing Script (lesson12_cleanRL_sac_run_test.py):** A lightweight script that reconstructs the *Agent*, loads the saved weights (.cleanrl_model), and runs a rendered episode to visually evaluate the trained policy.

Today Assignment

- In today's lesson, we will implement the update rule of the **SAC** algorithm within the *cleanRL* framework and we will run it on some gym environments. **On which environments can the SAC algorithm be used? Check the tables in the next slides.**
- Your task is to fill in the code where there are `#TODO` sections.

SAC to implement (<https://spinningup.openai.com/en/latest/algorithms/sac.html>)

Algorithm 1 Soft Actor-Critic

- 1: Input: initial policy parameters θ , Q-function parameters ϕ_1, ϕ_2 , empty replay buffer $\mathcal{D}$
- 2: Set target parameters equal to main parameters $\phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$
- 3: **repeat**
- 4: Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
- 5: Execute a in the environment
- 6: Observe next state s' , reward r , and done signal d to indicate whether s' is terminal
- 7: Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
- 8: If s' is terminal, reset environment state.
- 9: **if** it's time to update **then**
- 10: **for** j in range(however many updates) **do**
- 11: Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
- 12: Compute targets for the Q functions:

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s')$$

- 13: Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

- 14: Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s) \right),$$

where $\tilde{a}_\theta(s)$ is a sample from $\pi_\theta(\cdot|s)$ which is differentiable wrt θ via the reparametrization trick.

- 15: Update target networks with

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2$$

- 16: **end for**
- 17: **end if**
- 18: **until** convergence

SAC theory

Soft Actor Critic (SAC) is an **off-policy** algorithm that optimizes a stochastic policy in an off-policy way, forming a bridge between stochastic policy optimization and DDPG-style approaches. A central feature of SAC is **entropy regularization**. The policy is trained to maximize a trade-off between expected return and entropy, a measure of randomness in the policy.

- The entropy H of a stochastic variable x can be computed from its distribution P according to $H(P) = E_{x \sim P}[-\log P(x)]$.
- In entropy-regularized reinforcement learning, the agent gets a bonus reward at each time step proportional to the entropy of the policy at that timestep. This changes the RL problem to:

$$\pi^* = \arg \max_{\pi} E_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot | s_t)) \right) \right], \quad (1)$$

where $\alpha > 0$ is the trade-off coefficient.

Value and Q-functions are also changed to include the entropy bonuses

$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(R(s_t, a_t, s_{t+1}) + \alpha H(\pi(\cdot | s_t)) \right) \middle| s_0 = s \right], \quad (2)$$

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) + \alpha \sum_{t=1}^{\infty} \gamma^t H(\pi(\cdot | s_t)) \middle| s_0 = s, a_0 = a \right], \quad (3)$$

V^{π} and Q^{π} are connected by:

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi} [Q^{\pi}(s, a) + \alpha H(\pi(\cdot | s))] \quad (4)$$

SAC theory

SAC concurrently learns a policy π_θ and two Q-functions Q_{ϕ_1} , Q_{ϕ_2} (no V function is used). CleanRL implements a version with a fixed entropy regularization coefficient α , but the entropy-constrained variant is generally preferred by practitioners.

- Both Q-functions are learned with Mean Squared Bellman Error (MSBE) minimization, by regressing to a single shared target;
- The shared target is computed using target Q-networks, and the target Q-networks are obtained by polyak averaging the Q-network parameters over the course of training;
- The shared target makes use of the clipped double-Q trick (introduced in TD3).
- Full math details at <https://spinningup.openai.com/en/latest/algorithms/sac.html>

SAC theory: Q-function loss

The loss functions for the Q-networks in SAC are:

$$L(\phi_i, \mathcal{D}) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(Q_{\phi_i}(s, a) - y(r, s', d) \right)^2 \right], \quad (5)$$

where the target is given by

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{j=1,2} Q_{\phi_{\text{targ},j}}(s', \tilde{a}') - \alpha \log \pi_{\theta}(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_{\theta}(\cdot|s'). \quad (6)$$

SAC theory: policy loss

The policy should, in each state, act to maximize the expected future return plus expected future entropy. That is, it should maximize $V^\pi(s)$, which can be expanded out into

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a) + \alpha H(\pi(\cdot|s))] \quad (7)$$

Which is

$$V^\pi(s) = \mathbb{E}_{a \sim \pi} [Q^\pi(s, a) - \alpha \log \pi(a|s)]. \quad (8)$$

The way the policy is optimized makes use of the **reparameterization trick**, in which a sample from $\pi_\theta(\cdot|s)$ is drawn by computing a deterministic function of state, policy parameters, and independent noise.

The policy is then optimized according to

$$\max_{\theta} \mathbb{E}_{s \sim \mathcal{D}, \xi \sim \mathcal{N}} \left[\min_{j=1,2} Q_{\phi_j}(s, \tilde{a}_\theta(s, \xi)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s, \xi)|s) \right] \quad (9)$$

SAC: Implementation

- 1 The **CleanRL** implementation is centralized. Unlike other frameworks, networks, buffers, and the update logic coexist in an explicit and straightforward manner.
 - 2 At every valid step, a batch is sampled, and the update is performed in four sequential phases:
-

```
# UPDATE Q-NETWORK (CRITICS)  
# TODO: Implement Q-function optimization
```

```
# UPDATE POLICY (ACTOR)  
# TODO: Implement Policy optimization
```

```
# UPDATE ALPHA (TEMPERATURE)  
# TODO: Implement Alpha tuning
```

```
# UPDATE TARGET NETWORKS  
# TODO: Implement target network updates
```

SAC: Implementation - Q-function Loss

- 3 The Critic's loss minimizes the error between its prediction and the **Soft Bellman Backup**, which includes an entropy term.
- 4 We compute the empirical target $y = r + \gamma(1 - d)(\min Q_{\text{target}} - \alpha \log \pi)$ by blocking the gradients:

```
with torch.no_grad():  
    next_state_actions, next_state_log_pi, _ = actor.get_action(b_next_obs)  
    qf_next_target = torch.min(  
        qf1_target(b_next_obs, next_state_actions),  
        qf2_target(b_next_obs, next_state_actions)  
    ) - alpha * next_state_log_pi  
    next_q_value = b_rewards + (1 - b_dones) * gamma * qf_next_target
```

- 5 We compute the MSE Loss for both twin critics:

```
qf1_loss = F.mse_loss(qf1(b_obs, b_actions), next_q_value)  
qf2_loss = F.mse_loss(qf2(b_obs, b_actions), next_q_value)
```

SAC: Implementation - Policy Loss

- 6 The Actor learns a stochastic policy $\pi_{\theta}(a|s)$ to maximize expected reward plus entropy.
- 7 Using the **reparameterization trick**, we sample actions and pass them to the Critics:

```
# Sample action via Reparameterization
actions, log_pi, _ = actor.get_action(b_obs)
qf_pi = torch.min(qf1(b_obs, actions), qf2(b_obs, actions))

# Loss = Entropy - Value
actor_loss = ((alpha * log_pi) - qf_pi).mean()
```

- 8 At the end of the cycle, we perform a Polyak τ **Soft Update** to the twin target networks:

```
for param, target_param in zip(qf.parameters(), qf_target.parameters()):
    target_param.data.copy_(tau * param.data + (1 - tau) * target_param.data)
```

Gymnasium environments

Environment	States	Actions	Transition
MountainCar-v0	2 Continuous	3 Discrete	Deterministic
CartPole-v1	4 Continuous	2 Discrete	Deterministic
Acrobot-v1	6 Continuous	3 Discrete	Deterministic
LunarLander-v2	8 Continuous	4 Discrete	Deterministic
MountainCarContinuous-v0	2 Continuous	1 Continuous	Deterministic
Pendulum-v0	3 Continuous	1 Continuous	Deterministic
BipedalWalker-v3	24 Continuous	4 Continuous	Deterministic
FrozenLake-v0	16 Discrete	4 Discrete	Stochastic

How can we make SAC work on discrete state spaces? Can it work on discrete action spaces?

RL Algorithm capabilities

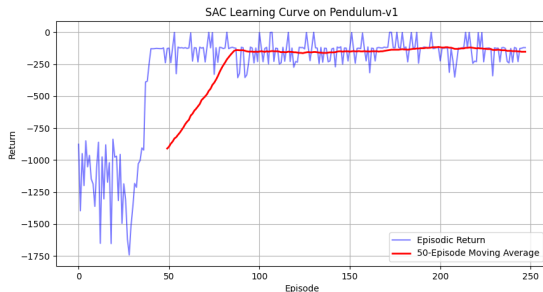
Algorithm	States	Actions	Transition	On/Off-policy	V/Q	π
Policy gradient	D	D	Needed	Planning	V	No
MC Control	D	D	No	On-policy	Q	No
Sarsa	D	D	No	On-policy	Q	No
Q-Learning	D	D	No	Off-policy	Q	No
Dyna-Q	D	D	Learned	Off-policy	Q	No
DQN	D/C	D	No	Off-policy	Q(2)	No
REINFORCE	D/C	D/C	No	On-policy	No	Yes
REINFORCE+B	D/C	D/C	No	On-policy	V	Yes
A2C	D/C	D/C	No	On-policy	V	Yes
MCTS	D	D	Needed	Planning	Q	No
PPO	D/C	C/D	No	On-policy	V	Yes
DDPG	D/C	C	No	Off-policy	Q	Yes
SAC	D/C	C/D	No	Off-policy	Q	Yes

Expected Results - Pendulum-v1 with SAC

Once the algorithm is complete it can be run with the command

```
python lesson12\_cleanRL\_sac\_test\_sac.py --env Pendulum-v1 --algo sac --exp\_name fi
```

It produces textual output in the terminal and a chart with performance over training episodes



Expected Results - Pendulum-v1 with SAC

When the training concludes, check the newly created experiment folder (e.g., `runs/Pendulum-v1__SAC__.../`). You will find:

- **Learning Curve (.png):** A pre-generated plot showing the episodic return over the training steps (raw and smoothed moving average).
- **Model Weights (.pth):** A PyTorch state dictionary containing the trained Actor's weights. To visualize your trained agent playing the game, use the test scripts:

```
.\run_test.bat    # Windows  
./run_test.sh     # Unix / macOS
```

Training on other environments

Compare the performance of SAC, DDPG, PPO and VPG on all the environments in the table above for which the algorithms can be compared. E.g., how does it perform on Pendulum-v0?

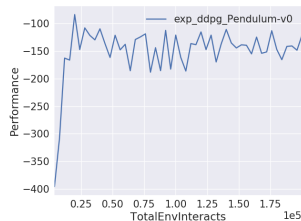


Figure: Mean return for DDPG on Pendulum-v0.

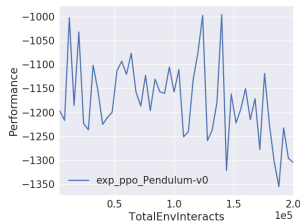


Figure: Mean return for PPO on Pendulum-v0.

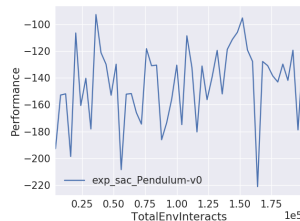


Figure: Mean return for SAC on Pendulum-v0.

Try to change the parameters of the algorithm and analyze the effect on the performance.

Interesting questions

- Does the code work with discrete state spaces? What can you change to make it work?
- Can you run the algorithms also on other gym environments? How?